Actividad en clases: TensorFlow Playground

El propósito de esta actividad es explorar visualmente el funcionamiento de las redes neuronales y comprender cómo su arquitectura, funciones de activación y parámetros de entrenamiento afectan su capacidad para aprender y generalizar.

Utilizaremos **TensorFlow Playground**, una herramienta interactiva que permite construir y entrenar redes neuronales sin necesidad de programarlas.

Instrucciones generales

- Trabaje en TensorFlow Playground, seleccionando el problema de **clasificación (Classification)**.
- Observe tanto las fronteras de decisión como los valores de *training loss* y *test loss*.
- Al comparar configuraciones, procure modificar solamente una característica a la vez y reiniciar el entrenamiento.
- Salvo que se indique lo contrario, utilice la función de activación **tanh**, sin regularización y con un learning rate de 0.03.
- No es necesario alcanzar un valor de pérdida específico. Lo importante es observar el comportamiento de las redes y explicar los resultados.

Recuerde que el entrenamiento depende también de la inicialización aleatoria de los pesos, por lo que una misma arquitectura puede producir resultados diferentes.

Parte A. Problema XOR: ¿por qué necesitamos redes neuronales?

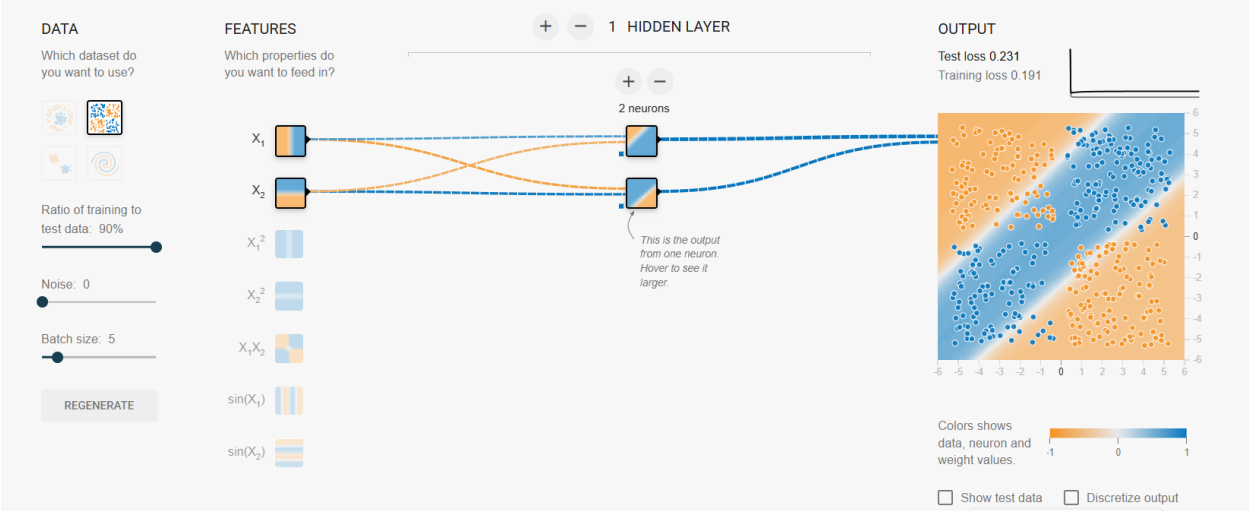
El problema XOR consiste en clasificar puntos de acuerdo con los signos de sus coordenadas:

- Clase $+1$ (azul): x_1 y x_2 tienen el mismo signo.
- Clase -1 (naranjo): x_1 y x_2 tienen signos opuestos.

Experimento 1: Modelo lineal

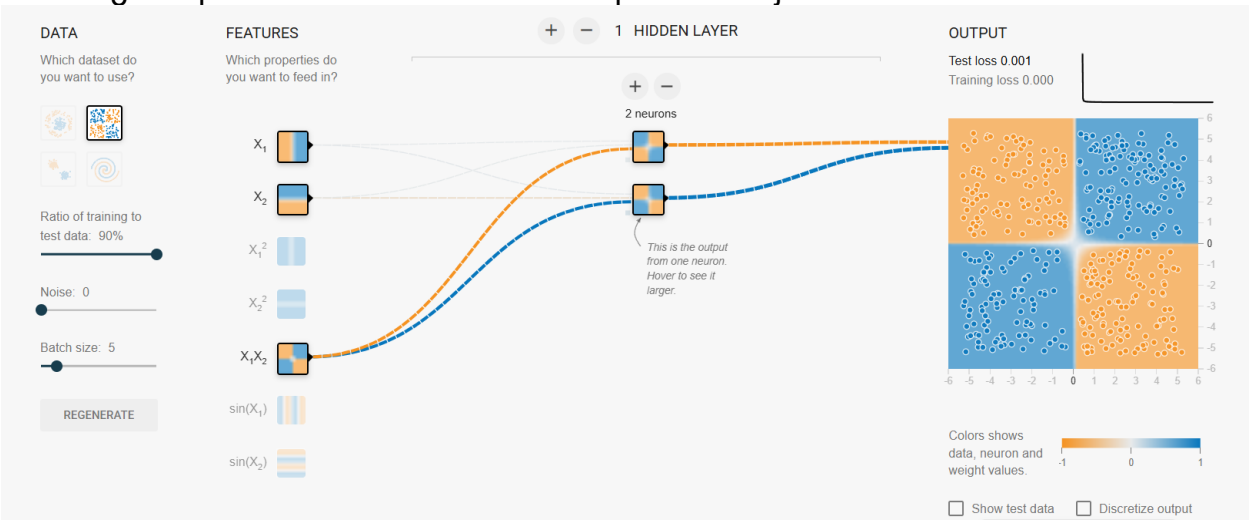
Seleccione el dataset XOR y utilice solamente las features x_1 y x_2.

1. Construya un modelo sin capas ocultas y entrénelo. ¿Consigue separar correctamente las dos clases? Explique por qué.



No, el modelo no consigue separar correctamente las dos clases. Esto ocurre porque solamente utiliza las variables x1 y x2, por lo que genera una frontera de decisión lineal. El problema XOR necesita separar los puntos que están en esquinas opuestas, y una sola línea no es suficiente para conseguirlo

2. Agregue la feature x_1x_2, manteniendo el modelo sin capas ocultas. ¿Qué ocurre ahora? ¿Por qué esta nueva característica permite mejorar la clasificación?



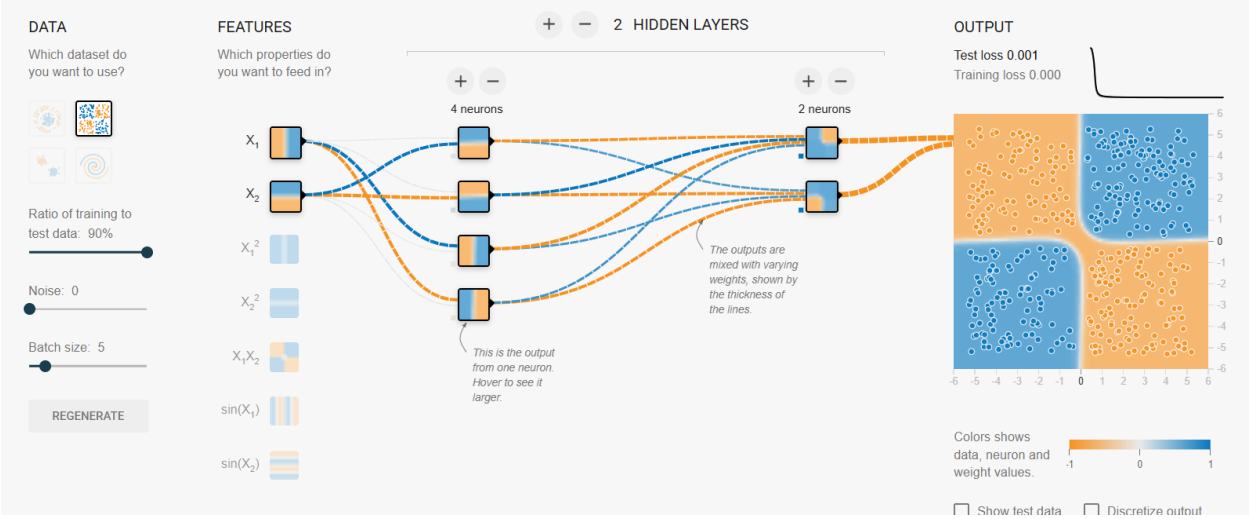
Al agregar la característica x1*x2, la clasificación mejora bastante. Esto sucede porque esta nueva variable entrega información sobre la relación entre x1 y x2 y

permite distinguir mejor los grupos. De esta manera, el modelo consigue separar las clases con mucha mayor precisión

Experimento 2: Capas ocultas y funciones de activación

Vuelva a seleccionar solamente x_1 y x_2.

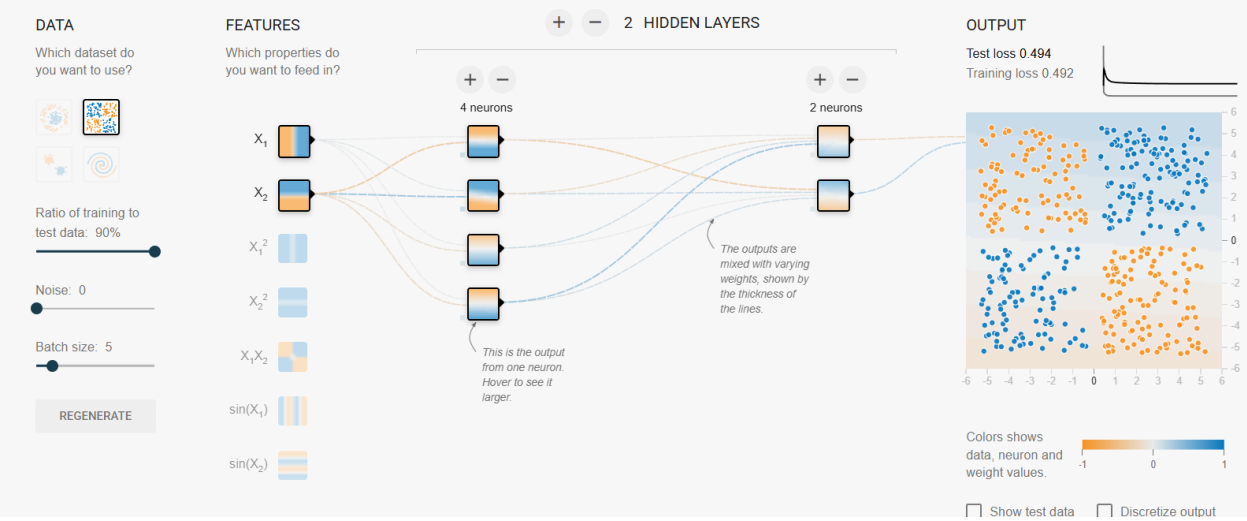
3. Agregue una capa oculta con función de activación $\tanh$. Experimente con distintas cantidades de neuronas hasta encontrar una arquitectura sencilla que consiga separar correctamente las clases. ¿Cuántas neuronas necesitó? Registre los valores de *training loss* y *test loss*.



Utilicé una red con 4 neuronas en la primera capa oculta y 2 en la segunda. Con esta configuración conseguí separar correctamente las dos clases. El training loss fue de 0.000 y el test loss fue de 0.001, por lo que el resultado fue bastante bueno.

4. Manteniendo esa arquitectura, cambie la función de activación a **Linear** y vuelva a entrenar. ¿Qué ocurre con la frontera de decisión? ¿Por qué?

Al cambiar la función de activación a Linear, la clasificación empeoró bastante. El training loss aumentó a 0.492 y el test loss a 0.494. Esto ocurre porque la función Linear no permite aprender fronteras de decisión complejas. Aunque tengamos varias capas, al utilizar solamente funciones lineales la red sigue comportándose como un modelo lineal.



5. Compare los resultados de agregar la feature x_1x_2 con los de utilizar una capa oculta. ¿Qué diferencia existe entre ambas estrategias?

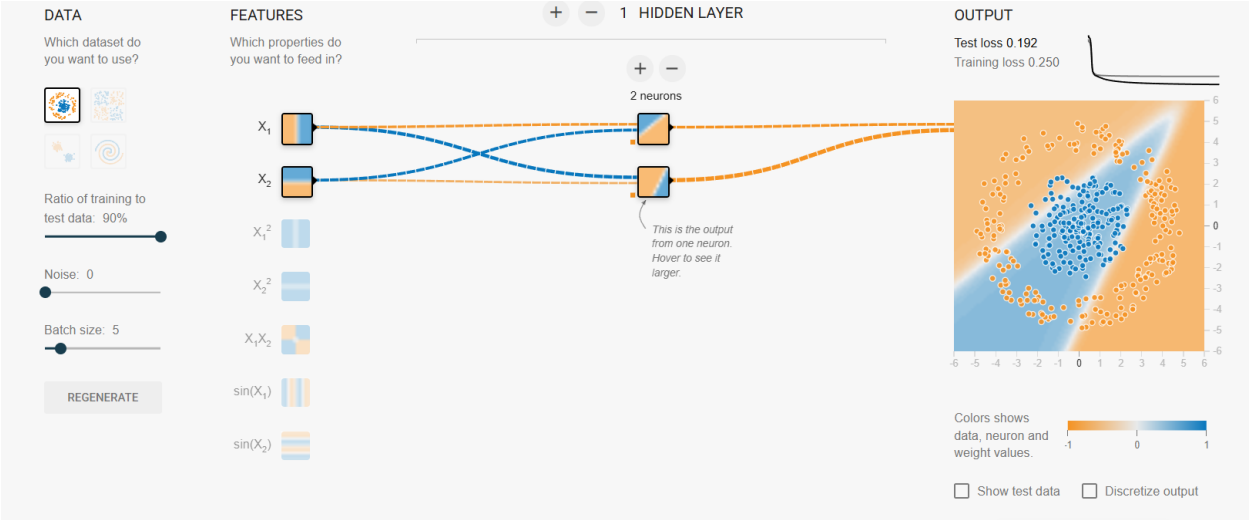
La diferencia es que al agregar x1*x2 estamos entregándole al modelo una característica nueva que nosotros creamos para facilitar la clasificación. En cambio, al utilizar una capa oculta, la propia red aprende a combinar las variables para encontrar una solución. Ambas estrategias ayudan a resolver el problema, pero funcionan de manera diferente.

Parte B. Complejidad de la red: círculos concéntricos

Seleccione el dataset de círculos concéntricos (**Circle**) y utilice solamente las features x_1 y x_2.

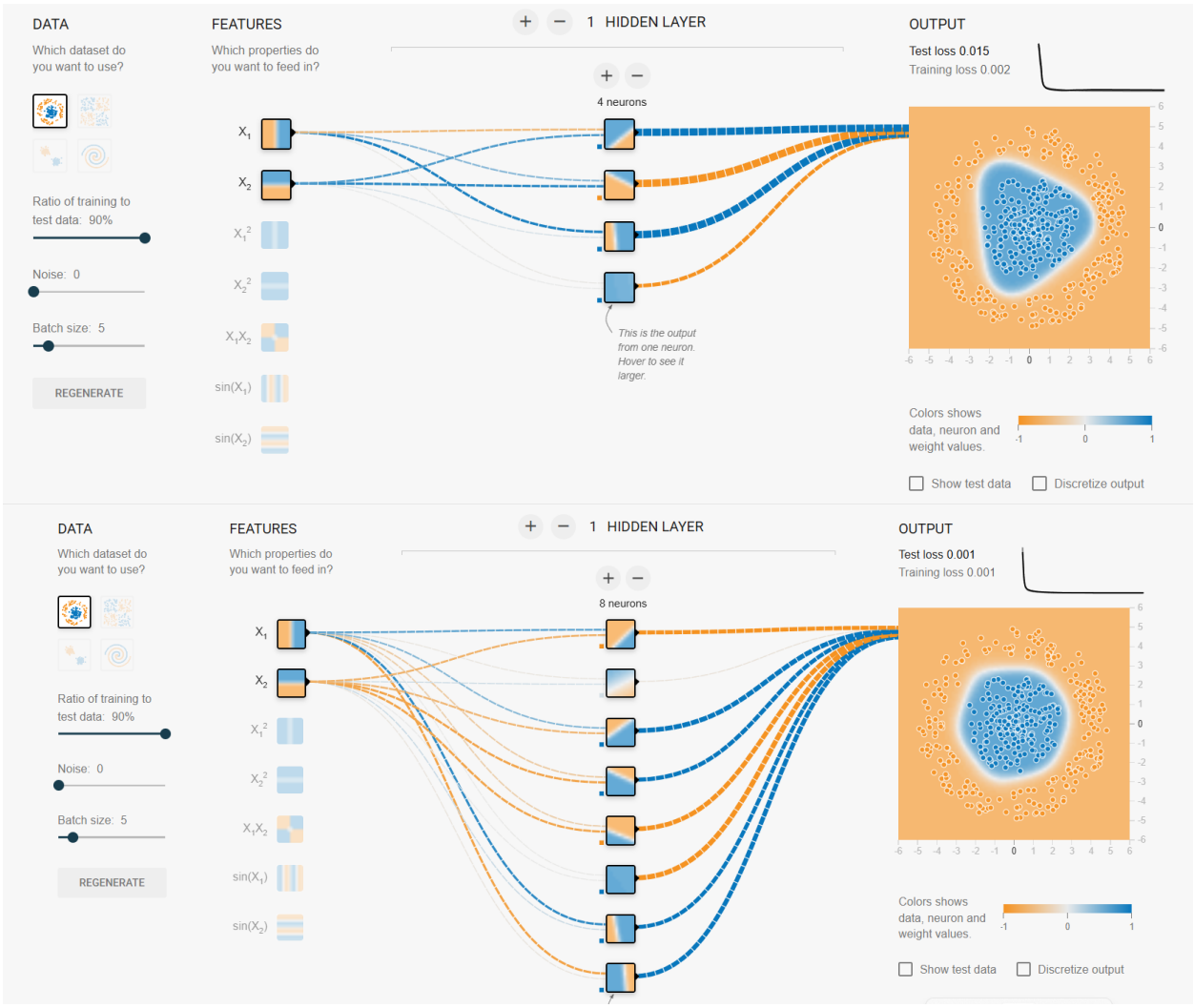
Experimento 3: Número de neuronas

1. Construya una red con una sola capa oculta y 2 neuronas. Entrénela y observe la frontera de decisión.



Con 2 neuronas, la red consigue reconocer parte de la distribución de los puntos, pero la frontera de decisión todavía no se ajusta bien a la forma circular. El training loss fue de 0.250 y el test loss de 0.192, por lo que todavía existen errores de clasificación.

2. Repita el experimento utilizando 4 y luego 8 neuronas, manteniendo los demás parámetros fijos. Compare los resultados después de un número similar de épocas de entrenamiento.



Al utilizar 4 neuronas, la clasificación mejoró bastante. El training loss fue de 0.002 y el test loss de 0.015. Después, al aumentar a 8 neuronas, obtuve un resultado todavía mejor, con un training loss de 0.001 y un test loss de 0.001.

3. ¿Cómo cambia la frontera de decisión al aumentar la cantidad de neuronas?

Al aumentar la cantidad de neuronas, la frontera de decisión se vuelve más detallada y consigue adaptarse mejor a la forma circular de los datos. Con 2 neuronas la separación era más sencilla, mientras que con 4 y 8 neuronas la red logró reconocer mejor los círculos

4. ¿Agregar más neuronas siempre produce una mejor clasificación? Compare los valores de *training loss* y *test loss*.

No necesariamente. En este experimento, al aumentar las neuronas mejoraron los resultados, pero eso no significa que siempre ocurra. Si agregamos demasiadas neuronas, la red podría aprender demasiado bien los datos de entrenamiento y cometer más errores con datos nuevos. Esto se conoce como overfitting.

5. ¿Qué relación existe entre la cantidad de neuronas y el poder expresivo (*expressive power*) de una red neuronal?

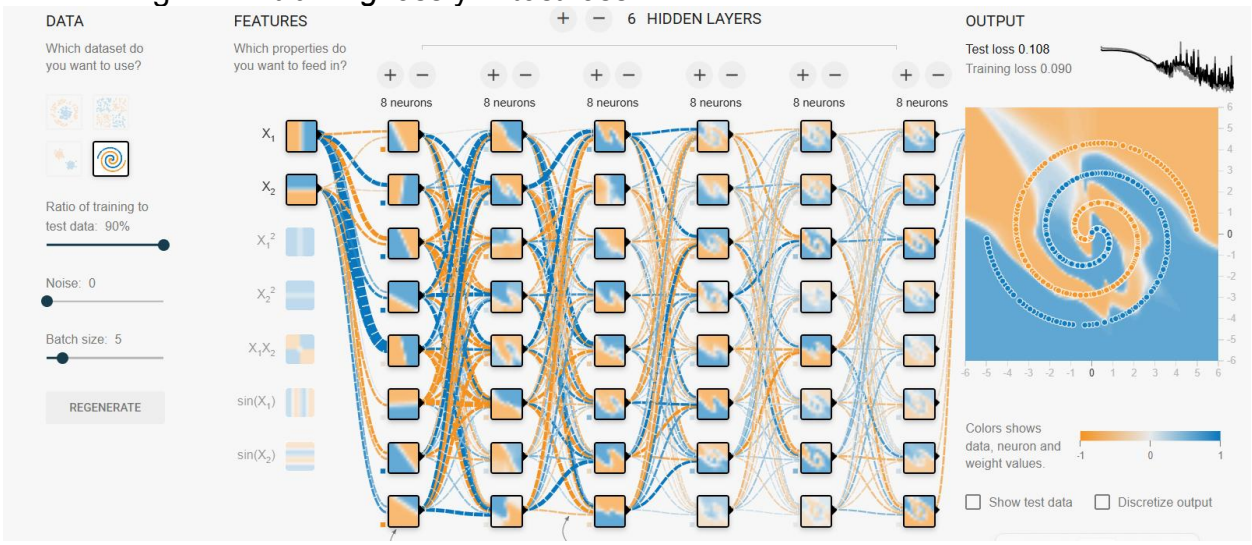
Mientras más neuronas tiene una red, mayor es su capacidad para aprender patrones y formas más complejas.

Parte C. Un problema más complejo: espiral

Seleccione el dataset **Spiral**, que presenta una frontera de clasificación considerablemente más compleja.

Experimento 4: Arquitectura y selección de features

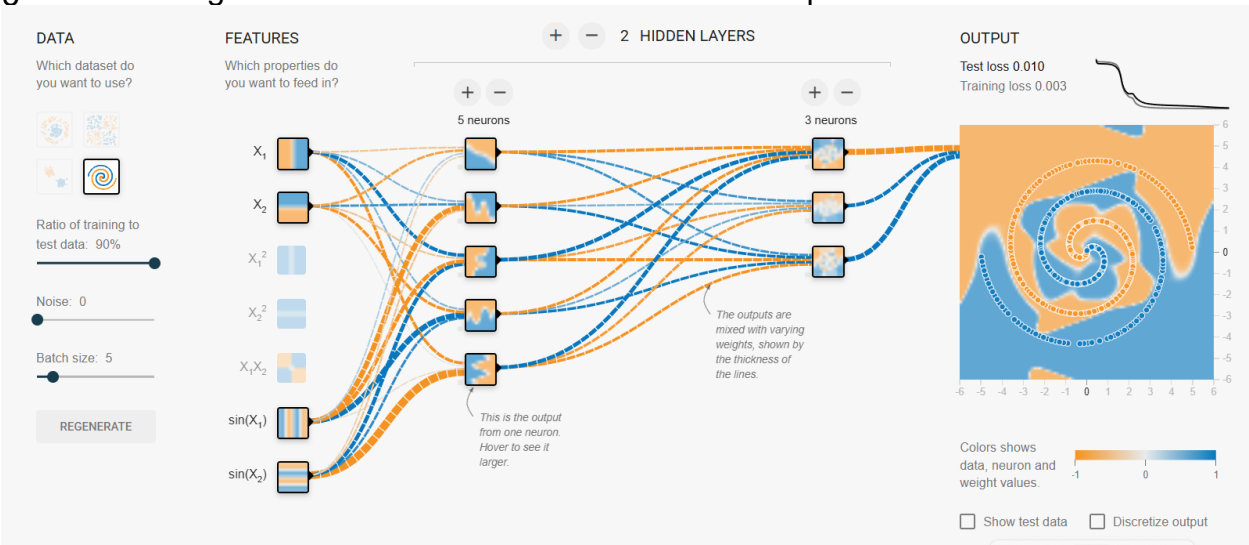
1. Utilizando solamente las features x_1 y x_2, experimente con el número de capas ocultas y neuronas para obtener una clasificación razonable. ¿Qué arquitectura utilizó? Registre el *training loss* y el *test loss*.



Para este experimento utilicé una red con 6 capas ocultas y 8 neuronas en cada una. A pesar de tener una red bastante grande, la clasificación no fue completamente buena,

ya que el problema de las espirales es más difícil de resolver. El training loss fue de 0.090 y el test loss fue de 0.108.

2. Ahora active las features adicionales que ofrece Playground y vuelva a entrenar. ¿Puede conseguir resultados similares utilizando una arquitectura más sencilla?



Sí, al activar las características adicionales pude conseguir una clasificación mucho mejor utilizando una red más sencilla. En este caso utilicé 2 capas ocultas, con 5 neuronas en la primera y 3 en la segunda. El training loss fue de 0.003 y el test loss fue de 0.010.

3. Compare ambas soluciones. ¿Qué ventajas y desventajas observa al proporcionar características transformadas en lugar de utilizar solamente las variables originales?

La principal ventaja de utilizar características transformadas es que ayudan a la red a reconocer patrones más fácilmente, por lo que puede conseguir buenos resultados con una arquitectura más sencilla. Como desventaja, necesitamos seleccionar las características adecuadas, ya que no todas necesariamente ayudan a resolver el problema.

4. Examine las fronteras de decisión y los valores de pérdida en entrenamiento y test. ¿Observa señales de *underfitting* u *overfitting*? Justifique su respuesta.

En la primera configuración se observa que la red todavía no consigue aprender completamente el patrón de las espirales. Los valores de pérdida son relativamente altos y similares entre entrenamiento y test, por lo que se puede considerar que existe underfitting.

En la segunda configuración los valores de pérdida son mucho más bajos. El training loss fue de 0.003 y el test loss de 0.010. Aunque existe una pequeña diferencia entre ambos, no se observa una señal importante de overfitting, ya que la red también consigue clasificar correctamente los datos de prueba.

Nota: No es necesario conseguir una clasificación perfecta. El objetivo es investigar cómo las características de entrada y la arquitectura afectan el aprendizaje.

Parte D. Entrenamiento: efecto del learning rate

Experimento 5: ¿Un learning rate mayor significa aprender más rápido?

Seleccione nuevamente el dataset de círculos y utilice una arquitectura que haya funcionado razonablemente bien en la Parte B.

Mantenga fija la arquitectura, las features y la función de activación.

1. Entrene la red utilizando tres valores de learning rate: 0.003, 0.03 y 0.3.

Para este experimento utilicé los tres valores indicados, manteniendo la misma arquitectura y las mismas características para poder comparar los resultados.

2. Reinicie el entrenamiento en cada caso y observe la evolución de la función de pérdida durante un número similar de épocas.

3. ¿Qué diferencias observa en la rapidez con que disminuye la pérdida?

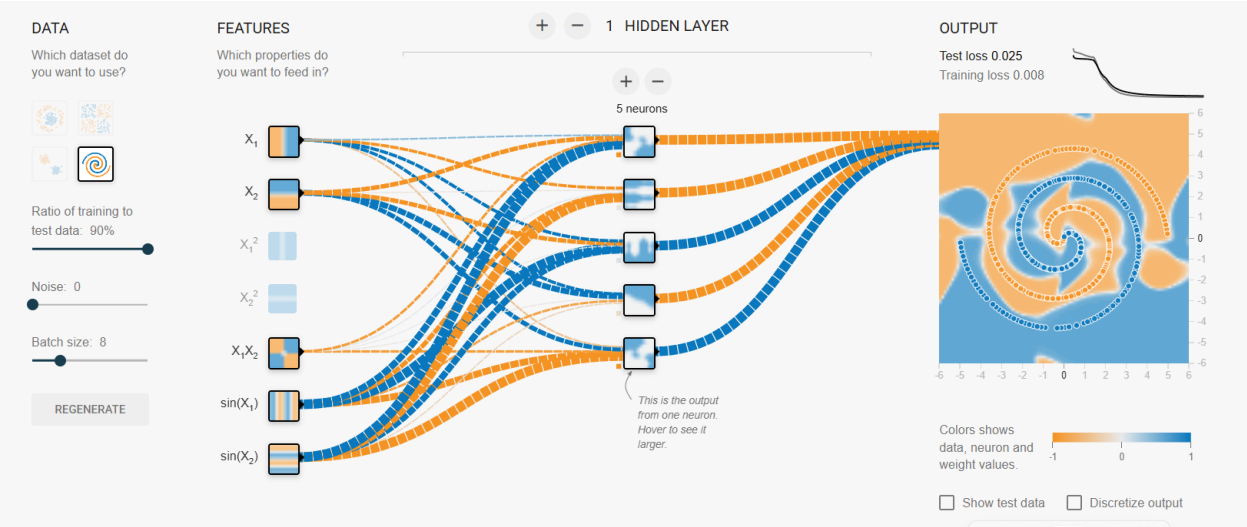
Con un learning rate de 0.003 el aprendizaje es más lento. Con 0.03 la pérdida disminuye más rápidamente y de forma más equilibrada. Con 0.3 los cambios pueden ser mucho más bruscos

4. ¿Un learning rate mayor produce siempre mejores resultados? ¿Qué problemas pueden aparecer cuando es demasiado grande?

No, un learning rate mayor no siempre produce mejores resultados. Si es demasiado grande, la red puede realizar cambios muy bruscos en sus parámetros y pasar de largo el valor que permite minimizar el error

Desafío opcional

Explore si es posible mejorar significativamente la clasificación con un modelo sin capas ocultas utilizando solamente dos de las features transformadas disponibles x_1^2, x_2^2, $x_1 x_2$, $\sin(x_1)$, $\sin(x_2)$.



¿Qué combinaciones probó? ¿Qué aprendió sobre la importancia del *feature engineering*?

Probé utilizando las características originales y algunas transformadas, como x1*x2, sin(x1) y sin(x2). En mi configuración utilicé una capa oculta con 5 neuronas, una función de activación Sigmoid, un learning rate de 0.3 y un batch size de 8.

Con esto s ve que agregar características transformadas puede ayudar a la red a reconocer patrones más complejos.

Conclusiones

A partir de los experimentos realizados, discuta brevemente:

1. ¿Por qué son importantes las funciones de activación no lineales en las redes neuronales?

   Porque permiten que la red aprenda patrones más complejos y no solamente relaciones lineales

2. ¿Una red con más capas o neuronas necesariamente aprende mejor?

   No, tener más neuronas no siempre significa mejores resultados. Lo importante es encontrar una red adecuada para cada problema

3. El Teorema del Aproximador Universal establece que una red neuronal suficientemente grande puede aproximar funciones continuas con una precisión arbitraria. ¿Significa esto que siempre podremos entrenar exitosamente esa red? Explique.

   No, porque también depende de los datos y de cómo se entrene la red. Tener muchas neuronas no garantiza que aprenda correctamente.

Registro de resultados

Para cada parte, registre las configuraciones utilizadas, los resultados más relevantes y sus conclusiones. Incluya capturas de pantalla de las soluciones que considere más interesantes y el enlace de Playground correspondiente, que permite compartir la configuración de la red.

No es necesario incluir todas las pruebas realizadas, solamente las necesarias para justificar sus respuestas.